

- 裸金属 IDC 集群 ingress-nginx 入口负载均衡 —— 两方案对比
 - 0. 一句话结论
 - 1. 统一规划参数(两方案共用)
 - 2. 两条全链路对照
 - 3. 方案一 —— 边缘双机 Keepalived(VRRP)+ HAProxy 四层负载
 - 3.1 前置确认(先做,否则白干)
 - 3.2 为什么一个公网 IP 就够(跨网卡 VIP 机制)
 - 3.3 Keepalived 配置
 - 3.4 HAProxy 配置
 - 3.5 ingress-nginx 安装(方案一)
 - 3.6 TLS 变体(可选:边缘终止)
 - 3.7 故障切换演示
 - 3.8 优缺点
 - 4. 方案二 —— 公网放路由器 + Calico BGP-LB(路由器 ECMP),无边缘 nginx
 - 4.1 拓扑
 - 4.2 装 Calico BGP-LB
 - 4.3 ingress-nginx 安装(方案二)
 - 4.4 路由器侧配置(通用 IOS-like 伪配)
 - 4.5 公网 IP 放在路由器(无边缘 nginx)
 - 4.6 故障切换演示
 - 4.7 优缺点
 - 5. 并排 Trade-off 对比
 - 6. 选型建议 + 落地路径
 - 7. 验证清单

裸金属 IDC 集群 ingress-nginx 入口负载均衡 —— 两方案对比

场景:7 节点裸金属 K8s 集群在 IDC 机房,公网 IP 现在落在**单台 nginx**上(单点 + 没分流)。目标:多 ingress 节点真分流 + 入口高可用,消除单点。本文一篇写全两个方案,所有配置可直接照抄落地。

0. 一句话结论

你的情况	选	为什么
路由器暂不可控、想最快上线	方案一: 边缘 nginx 双机 Keepalived + HAProxy	纯集群外,不依赖 BGP;公网落边缘
有可控路由器、要可扩展/长期生产标准	方案二: 公网放路由器 + Calico BGP-LB ECMP	路由器 DNAT+ECMP, 无边缘 nginx ;先单机后双机
路由器还没就绪	先方案一兜底,BGP 就绪再切方案二	见第 6 节

(前提需要部署ingrssl-nginx下沉到k8s节点中 配置好相关的业务nginx路由配置)。

1. 统一规划参数(两方案共用)

项	值
集群节点 (192.168.100.0/24)	master×3:k8smaster1 .10 / k8smaster3 .21 / k8smaster2 .30;worker×4:k8swork1 .11 / k8swork2 .12 / k8swork3 .27 / k8swork4 .28
ingress 节点(打标签 ingress=true)	k8swork1 .11、k8swork2 .12(后续可扩展 .27/.28);master 不承载入口流量
ingress 部署形态	DaemonSet +nodeSelector: ingress=true + hostNetwork: true,节点直接 bind :80/:443
边缘设备	nginx2(现有,持公网 IP)+ nginx2(新增), 双网卡 :eth0 公网 侧 / eth1 内网 192.168.100.x
边缘内网 IP	nginx2 =192.168.100.32,nginx1 = 192.168.100.xxx
公网 IP	116.211.238.197(你机房分配的那一个,全文用占位符)
集群 LB VIP 段(方案二)	192.168.100.200/29(.200-.207,8 个;ingress 用 .200)
BGP ASN	集群侧 64512 / 路由器侧 64513(私有段 64512–65534)
路由器 BGP 邻居 IP	192.168.100.25x(机房上联路由器,示例占位)
Pod CIDR	10.244.0.0/16

2. 两条全链路对照

方案一(负载在边缘 L4):

```
client —公网—> [公网VIP] nginx1/nginx2 (Keepalived主备)
                        |
                        | HAProxy mode tcp / balance leastconn
                        |—> k8swork1 .11:443 ┐
                        |—> k8swork2 .12:443 ┘ ingress-nginx(hostNetwork)→ svc →
```

pod

- 分流发生在:HAProxy → 多 ingress 节点
- HA 靠:Keepalived VRRP(公网 IP 在两台边缘漂)
- 真实客户端 IP:HAProxy send-proxy-v2 → ingress use-proxy-protocol

方案二(公网放路由器,负载在路由器 L3 ECMP):

```
client —公网—> [公网IP] 机房路由器(可控)
                        |
                        | DNAT/路由 公网 → 192.168.100.200
                        | 同一台路由器 BGP ECMP 多 nexthop
                        | 192.168.100.200 (Service LoadBalancer)
                        |—> k8swork1 .11 ┐
                        |—> k8swork2 .12 ┘ ingress-nginx(hostNetwork)→ svc → pod
```

- 无边缘 nginx、无 proxy_protocol
- 分流发生在:路由器 ECMP(内核级,按 5 元组 hash)
- HA 靠:BGP 撤路由(节点挂自动剔除)+ 路由器自身 HA(先单机过渡 → 后期堆叠/双机热备)
- 真实客户端 IP:externalTrafficPolicy: Local(client 直达节点,真实 IP 天然保留)

3. 方案一 —— 边缘双机 Keepalived(VRRP)+ HAProxy 四层负载

定位:负载均衡和入口 HA 都在集群外完成。集群内 ingress 纯 hostNetwork,Service 类型无所谓(边缘直连节点 :443)。不依赖路由器 BGP。

3.1 前置确认(先做,否则白干)

VRRP 切换 = 同一个公网 IP 从 nginx1 的网卡"漂"到 nginx2 的网卡,靠**免费 ARP(gratuitous ARP)**通告上联交换机更新 MAC 表。机房如果锁了 MAC,漂移不生效。

先问机房 / 自己验证:

检查	命令 / 话术
问机房	"这个公网 IP 能不能在我两台设备之间做 VRRP 漂移?上联端口有没有做 IP-MAC 绑定 / port-security / 静态 ARP?"
自验免费 ARP 能否生效	在 nginx2 上临时: <code>ip addr add <PUBLIC_IP>/<掩码> dev eth0; arping -c 3 -U -I eth0 <PUBLIC_IP></code> ,然后从外部 ping <PUBLIC_IP> 看是否切到 nginx2;验证完 <code>ip addr del</code> 撤掉

结论分支:

- 机房允许漂移 → 方案一成立,继续。
- 机房锁 MAC(常见于云厂商裸金属 / 严格 IDC)→ 方案一不可行,直接走方案二(BGP 由路由器宣告,不靠 ARP 漂移),或用机房提供的"浮动 IP / 弹性 IP API"做切换。

3.2 为什么一个公网 IP 就够(跨网卡 VIP 机制)

每台边缘设备两块网卡:



要点:

1. **VRRP 选举 / 心跳**绑在内网网卡 **eth1**(有固定 IP),两台用 `unicast_peer` 互发心跳——公网侧完全不参与选举,所以 eth0 平时没地址也无所谓。
2. **VIP 加在哪**:`virtual_ipaddress { <PUBLIC_IP> dev eth0 }`,谁当 master 就把公网 IP 临时挂到谁的 eth0;故障时摘掉、对端挂上。
3. **eth0 无地址也能工作**:网卡 link up 就能收发,IP 由 keepalived 动态挂;master 上 eth0 一旦有 VIP,内核就替它应答 ARP 并发免费 ARP。
4. **一个公网 IP = 主备(active-passive)**:同时只有一台对外。真正的"负载均衡"发生在 master 上 HAProxy → 多个 ingress 节点这一跳,不受主备影响。(要边缘双活需 2 个公网 IP + DNS 轮询,这里用不上。)

3.3 Keepalived 配置

两台都装:`yum install -y keepalived` 或 `apt install -y keepalived`。

nginx1(MASTER)/etc/keepalived/keepalived.conf:

```
global_defs {
    router_id nginx1
    enable_script_security
    script_user root
}

# HAProxy 进程挂了就降优先级,把 VIP 让给对端
vrrp_script chk_haproxy {
    script "/usr/bin/killall -0 haproxy" # 进程存活探测
    interval 2
    weight -40 # 挂了 priority 100-40=60 < 90,触发切换
    fall 2
    rise 2
}

vrrp_instance VI_PUB {
    state MASTER
    interface eth1 # ← 选举/心跳走内网网卡(有固定 IP)
    virtual_router_id 51 # 两台必须一致;同网段内唯一
    priority 100 # MASTER 高
    advert_int 1
    unicast_src_ip 192.168.100.241 # 本机内网 IP
    unicast_peer {
        192.168.100.242 # 对端(nginx2)内网 IP
    }
    authentication {
        auth_type PASS
        auth_pass Edg3VrrP # 两台一致,自定义
    }
    virtual_ipaddress {
        <PUBLIC_IP>/<掩码> dev eth0 # ← VIP 挂到公网网卡;平时 eth0 无 IP
    }
    track_script {
        chk_haproxy
    }
}
```

nginx2(BACKUP) 只改 3 处:

```
router_id nginx2 # global_defs 里
state BACKUP
priority 90
```

```
unicast_src_ip 192.168.100.242
unicast_peer { 192.168.100.241 }
```

启动:`systemctl enable --now keepalived`。

内核需允许绑非本机 IP(HAProxy 绑 VIP 时):`echo`

`'net.ipv4.ip_nonlocal_bind=1' >> /etc/sysctl.conf && sysctl -p`。

3.4 HAProxy 配置

两台都装:`yum install -y haproxy` / `apt install -y haproxy`。两台配置完全相同(谁是 master 谁干活)。

`/etc/haproxy/haproxy.cfg`:

```
global
    log /dev/log local0
    maxconn 100000
    nbthread 4
    daemon

defaults
    mode tcp
    timeout connect 5s
    timeout client 50s
    timeout server 50s
    option dontlognull
    retries 2

# ----- HTTP :80 -----
frontend ft_http
    bind *:80
    default_backend bk_ingress_http

backend bk_ingress_http
    balance leastconn
    # send-proxy-v2 把真实客户端 IP 透传给 ingress(配套 3.5)
    server k8swork1 192.168.100.11:80 check send-proxy-v2
    server k8swork2 192.168.100.12:80 check send-proxy-v2

# ----- HTTPS :443(TLS 透传,ingress 终止)-----
frontend ft_https
    bind *:443
    default_backend bk_ingress_https

backend bk_ingress_https
    balance leastconn
    option ssl-hello-chk          # 用 TLS ClientHello 探活后端
    server k8swork1 192.168.100.11:443 check send-proxy-v2
```

```
server k8swork2 192.168.100.12:443 check send-proxy-v2

# ----- 可选:状态页 -----
listen stats
    mode http
    bind 192.168.100.241:9000          # 各机填自己内网 IP
    stats enable
    stats uri /
    stats refresh 5s
```

启动:`systemctl enable --now haproxy`。

关键:HAProxy `mode tcp` 是四层透传,TLS 在 ingress-nginx 上终止(证书放 K8s Secret,不放边缘)。后端要扩节点,直接在 backend 加 `server` 行即可。

3.5 ingress-nginx 安装(方案一)

hostNetwork 模式,Service 只要 ClusterIP(边缘直连节点端口,不需要 LoadBalancer):

```
# 打标签 + 装
bash install.sh --label-nodes=k8swork1,k8swork2 --service-type=ClusterIP
```

让 ingress 认 `proxy-protocol`(收 HAProxy 的 `send-proxy-v2`,才能拿到真实客户端 IP),编辑 ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ingress-nginx-controller
  namespace: ingress-nginx
data:
  use-proxy-protocol: "true"
  use-forwarded-headers: "true"
  compute-full-forwarded-for: "true"
```

```
kubectl -n ingress-nginx rollout restart ds/ingress-nginx-controller
```

⚠ `use-proxy-protocol: "true"` 后,所有到达 ingress 的连接都必须带 proxy-protocol 头。即只能经 HAProxy 进,不能再直接 `curl` 节点 IP:80(会握手失败)。测试要打公网 VIP。

3.6 TLS 变体(可选:边缘终止)

若想在边缘卸载 TLS(证书放 nginx/HAProxy):把 `ft_https` 改 `mode http + bind *:443 ssl crt /etc/haproxy/certs/site.pem`,后端走 `:80`。默认不推荐——证书分散到边缘、ingress 的 annotation/重定向能力用不上。保持 3.4 的 TCP 透传。

3.7 故障切换演示

```
# 在 master(nginx1)上停 HAProxy
systemctl stop haproxy
# 1~3s 后 VIP 应漂到 nginx2
ssh nginx2 'ip addr show eth0 | grep <PUBLIC_IP>' # 应能看到 VIP
# 外部持续 curl,观察是否仅极短中断
```

3.8 优缺点

✓ 优点	✗ 缺点
不依赖机房路由器,半天上线	边缘 active-passive,单台 master 扛全部流量(够用但不是双活)
故障域清晰,排查直观(HAProxy stats 一目了然)	多一跳(边缘→节点),时延略增
后端加节点 = 改 backend 一行	依赖机房允许 ARP/MAC 漂移(前置确认)
七层能力可选(需要时切 mode http)	边缘是有状态网元,需自己运维 keepalived/haproxy

4. 方案二 —— 公网放路由器 + Calico BGP-LB(路由器 ECMP),无边缘 nginx

定位:公网 IP 配在机房可控路由器上,路由器 DNAT/路由到内网 LB VIP,再由同一台路由器三层 ECMP 分流(内核级、多节点真分流、零额外跳数)。Calico 的一个 BIRD 进程同时宣告 Pod CIDR + LoadBalancer Service IP,不需要 MetalLB。不需要 nginx1/nginx2,也不需要 proxy_protocol。

4.1 拓扑



Service(ingress-nginx) type=LoadBalancer → EXTERNAL-IP = 192.168.100.200
路由器对 192.168.100.200/32 装多条等价 nexthop → 按 5 元组 hash 分流到多节点

4.2 装 Calico BGP-LB

```
bash ../calico/bgp-lb/install.sh \  
--apiserver-host=192.168.100.10 \  
--my-asn=64512 \  
--lb-cidr=192.168.100.200/29 \  
--peer-asn=64513 \  
--peer-address=<ROUTER_IP>
```

脚本会落以下两个 CR(此处完整贴出,便于核对 / 手工 apply):

BGPConfiguration(集群 AS + node mesh + LB/External IP 宣告):

```
apiVersion: crd.projectcalico.org/v1  
kind: BGPConfiguration  
metadata:  
  name: default  
spec:  
  asNumber: 64512  
  nodeToNodeMeshEnabled: true  
  serviceLoadBalancerIPs:  
    - cidr: 192.168.100.200/29  
  serviceExternalIPs:  
    - cidr: 192.168.100.200/29
```

BGPPeer(指向机房上联路由器):

```
apiVersion: crd.projectcalico.org/v1  
kind: BGPPeer  
metadata:  
  name: upstream-router
```

```
spec:
  peerIP: <ROUTER_IP>
  asNumber: 64513
```

脚本还会在 `kube-system` 部署一个 `lb-assigner`, 自动给 `type=LoadBalancer` 的 Service 从 `192.168.100.200/29` 里分配 EXTERNAL-IP (Calico 本身不自动分配 LB IP)。

4.3 ingress-nginx 安装(方案二)

```
bash install.sh --label-nodes=k8swork1,k8swork2 --service-type=LoadBalancer
```

Service 关键字段(`externalTrafficPolicy: Local`):

```
apiVersion: v1
kind: Service
metadata:
  name: ingress-nginx-controller
  namespace: ingress-nginx
spec:
  type: LoadBalancer
  loadBalancerIP: 192.168.100.200      # 固定到 .200(也可由 lb-assigner 自动分)
  externalTrafficPolicy: Local        # 关键:保真实 IP + 只让有 ingress pod 的节点
宣告 VIP
ports:
  - name: http
    port: 80
    targetPort: 80
  - name: https
    port: 443
    targetPort: 443
```

为什么 `externalTrafficPolicy: Local`:

1. 保真实客户端 IP: 不做 SNAT, 后端 pod 看到真实源 IP。
2. 配合 ECMP 正确收敛: 只有实际跑着 `ingress pod` 的节点才会把 `192.168.100.200` 的健康检查通过、才被路由器纳入 ECMP nexthop。节点 drain → pod 走 → 该节点退出 ECMP, 流量不黑洞。

验证:

```
kubectl -n ingress-nginx get svc ingress-nginx-controller # EXTERNAL-IP 应为 192.168.100.200
```

4.4 路由器侧配置(通用 IOS-like 伪配)

对每个 ingress 节点建 BGP 邻居,并开 ECMP 多路径:

```
router bgp 64513
  bgp log-neighbor-changes
  maximum-paths 16 ! ← 开 ECMP,允许多条等价路径
  neighbor 192.168.100.11 remote-as 64512
  neighbor 192.168.100.12 remote-as 64512
  ! 后续扩节点再加 .27 / .28
  address-family ipv4 unicast
    neighbor 192.168.100.11 activate
    neighbor 192.168.100.12 activate
```

关键点:

- **maximum-paths 16**:不开的话路由器只选一条最优路径 = 退化成单节点,没有 ECMP 就没有真分流。
- 路由器会收到 **192.168.100.200/32**(LB)和 **10.244.0.0/16**(Pod CIDR)的宣告。

4.5 公网 IP 放在路由器(无边缘 nginx)

架构决定:公网 IP 配在**机房可控路由器**的 WAN 口,路由器 DNAT/路由公网 → 内网 LB VIP **192.168.100.200**,再由同一台路由器 BGP ECMP 分流到多节点。**不需要 nginx1/nginx2,也不需要 proxy_protocol**——client 直达节点,后端 pod 直接看到真实公网源 IP,只靠 **externalTrafficPolicy: Local**。

路由器侧两条(IOS-like 伪配,接 4.4 的 BGP 段一起配):

```
! 1) 公网入站 DNAT 到内网 LB VIP(公网 IP 配在 WAN 口)
ip nat inside source static tcp 192.168.100.200 80 116.211.238.197 80
ip nat inside source static tcp 192.168.100.200 443 116.211.238.197 443
! 2) 或三层直路由(若机房允许公网段直接路由进内网,省 NAT):
! ip route 116.211.238.197/32 192.168.100.200
```

入口 HA —— 先单机,后期升双机:

阶段	形态	单点情况
第一步 (now)	单台路由器 持公网 IP + DNAT + BGP	路由器是单点;但已比"单台 nginx"强(后端多节点真分流),可作过渡
第二步	双路由器/防火墙 HA: 堆叠(IRF/iStack) 或双机热备(VRRP/HRP),公网 IP 做 VIP	入口无单点

升双机时对集群侧的唯一影响:**堆叠/逻辑一台 → BGPPeer 配 1 条**(指堆叠管理 VIP);**双机独立(VRRP)→ BGPPeer 配 2 条**(分别指两台路由器)。ingress / Calico 其余配置不变。

4.6 故障切换演示

```
# drain 一个 ingress 节点
kubect1 drain k8swork1 --ignore-daemonsets --delete-emptydir-data
# 该节点 ingress pod 走 → externalTrafficPolicy:Local 健康检查失败 → BGP 撤 .200 路由
# 路由器上观察 nexthop 从 2 条变 1 条
# show ip route 192.168.100.200      ! 期望多 nexthop, drain 后少一个
# 恢复
kubect1 uncordon k8swork1
```

4.7 优缺点

✓ 优点

路由器内核级 ECMP,真 · 多节点同时分流

加节点零改动(节点起 ingress pod 自动入 ECMP)

单 BIRD 进程同时管 Pod CIDR + LB IP, 无需 MetalLB

与本仓库生产标准一致,可扩到几十节点;无边缘网元

✗ 缺点

依赖机房路由器配合(BGP + maximum-paths)

BGP/ECMP 排障门槛高(birdcl / show ip route)

ECMP 按 hash 分流,rehash 时长连接可能瞬间

入口 HA 取决于路由器(单台是单点,需升堆叠/双机热备消除)

5. 并排 Trade-off 对比

维度	方案一 Keepalived+HAProxy	方案二 Calico BGP-LB + ECMP
负载发生层级	L4,在边缘 HAProxy	L3,在路由器 ECMP(内核转发)
真·多节点同时分流	✅ (HAProxy 向多节点轮询)	✅ (路由器多 nexthop hash)
入口 HA 机制	VRRP(公网 IP 主备漂移)	BGP 撤路由剔节点 + 路由器 HA(堆叠/双机热备)
故障收敛时间	VRRP ~1-3s;后端探活 ~2× interval	BGP keepalive/holdtime,默认数秒(可调)
客户端真实 IP	send-proxy-v2 + use-proxy-protocol	externalTrafficPolicy:Local(无边缘,无需 proxy_protocol)
是否依赖机房路由器	❌ 不依赖(只需允许 ARP 漂移)	✅ 依赖(可控路由器 + BGP + maximum-paths)
额外网络跳数	+1(边缘→节点)	0(公网→路由器→节点,无额外代理跳)
横向扩展(加 ingress 节点)	改 HAProxy backend 加一行	节点起 pod 自动入 ECMP + 路由器加 neighbor
运维复杂度	低(进程级,stats 直观)	中高(BGP/BIRD 概念 + 路由器协同)
设备/IP 成本	2 台边缘 + 1 个公网 IP(现有)	可控路由器(先 1 台后 2 台)+ 公网 IP + 1 段内网 LB VIP,无边缘 nginx
单点风险	边缘 active-passive, master 扛全量	后端多节点无瓶颈;入口单点在路由器(升双机消除)

每格依据:跳数=链路图实测路径;收敛时间=各协议默认计时器(VRRP advert_int=1s×fall;BGP holdtime 默认 90s 但 BFD/短计时可压到秒级);扩展性=两套"加一个节点"实际改动量。

6. 选型建议 + 落地路径

明确推荐:

- **有可控路由器 → 方案二。** 公网放路由器, DNAT + BGP ECMP, **无边缘 nginx**; 内核级 ECMP 无瓶颈、加节点零改动、与集群生命周期绑定(pod 在哪流量到哪), 是裸金属 K8s 生产标准。
- **路由器暂不可控 / 机房 BGP 没落实 → 先方案一。** 边缘 nginx 双机兜公网入口, 不碰路由器, 半天能上。

方案二分两步走(入口 HA 渐进):

1. **第一步(now):** 公网 IP 配到路由器, 装 Calico BGP-LB, ingress Service 切 **LoadBalancer**, 路由器 DNAT + BGP ECMP。 **单台路由器过渡**——已比"单台 nginx"强(后端多节点真分流), 入口暂为单点。
2. **第二步:** 路由器升 **堆叠(IRF/iStack)或双机热备(VRRP/HRP)**, 公网 IP 做 VIP, 入口单点消除。集群侧只需把 BGPPeer 从 1 条改 2 条(双机独立时)或不变(堆叠)。

灰度 / 回滚: 先拿一个测试域名解析到公网 IP、验证 ECMP(`show ip route 192.168.100.200` 看多 nexthop), 再全量。回滚: 把公网 IP 临时切回原 nginx 设备即可, 集群侧不动。

7. 验证清单

方案一:

```
# 打公网 VIP(不能直连节点,因为开了 proxy-protocol)
curl -I -H 'Host: test.example.com' http://<PUBLIC_IP>
# HAProxy 分流 / 健康检查
echo "show stat" | socat stdio /run/haproxy/admin.sock # 或看 :9000 stats 页
# VIP 漂移
systemctl stop haproxy && ssh nginx2 'ip addr show eth0 | grep <PUBLIC_IP>'
```

方案二:

```
kubectl -n ingress-nginx get svc ingress-nginx-controller # EXTERNAL-
IP=192.168.100.200
kubectl -n calico-system exec ds/calico-node -- birdcl show protocols # BGP
```

Established

路由器侧(示例)

show ip route 192.168.100.200 ! 期望多 nexthop

kubect1 drain k8swork1 --ignore-daemonsets --delete-emptydir-data # 看 ECMP
收敛

kubect1 uncordon k8swork1